

前端开发工程师题库

职业技能培训配套练习题

本题库包含单选题、多选题和判断题，紧扣知识库内容，覆盖HTML5、CSS3、JavaScript、ES6+、React、前端工程化、性能优化与安全等核心知识点。

第1章 HTML5 语义化与文档结构

1. 【单选题】以下哪个标签最适合用于表示页面的主导航链接组？

- A. <div class="nav">
- B. <navigation>
- C. <nav>
- D. <menu>

答案：C

解析：<nav> 是 HTML5 专门定义导航链接组的语义标签，具有明确的语义含义，有利于 SEO 和屏幕阅读器解析。

2. 【单选题】在 HTML5 中，<article> 标签最适合用于以下哪种场景？

- A. 页面的页脚版权信息
- B. 一篇独立的博客文章
- C. 网站的 Logo 图片
- D. 页面的侧边广告栏

答案：B

解析：<article>

表示文档中独立、完整且可独立分发或复用的内容，如博客文章、新闻报道，脱离当前页面仍然有意义。

3. 【单选题】以下关于 <main> 标签的说法正确的是？

- A. 一个页面可以有多个 <main> 标签
- B. <main> 可以嵌套在 <article> 内部
- C. 每个页面只能有一个 <main> 标签
- D. <main> 标签没有语义含义

答案：C

解析：<main> 表示文档的主要内容，一个页面只能有一个，且不能嵌套在其他语义元素（如 article、aside、footer、header、nav）内部。

4. 【多选题】以下哪些属于 HTML5 新增的表单输入类型？

- A. email

- B. tel
- C. date
- D. color

答案：ABCD

解析：HTML5 新增了多种输入类型，包括 email（邮箱）、tel（电话）、date（日期）、color（颜色），还有 url、number、range、time、search 等。

5. 【判断题】使用 `<div>` 和 `<span>` 配合 `class` 可以实现与语义化标签完全相同的视觉效果，因此语义化标签并不重要。

答案：错误

解析：虽然视觉效果可以相同，但语义化标签对 SEO、可访问性（屏幕阅读器）和代码可维护性有重要意义，不能仅用 `<div>` 替代。

第2章 CSS 选择器与盒模型

6. 【单选题】以下哪个选择器的优先级最高？

- A. .nav .item a
- B. #header .logo
- C. div.container > p
- D. a:hover

答案：B

解析：#header .logo 优先级为 (1, 1, 0)。A 是 (0, 2, 1)，C 是 (0, 1, 1)，D 是 (0, 1, 1)。ID 选择器的权重远高于类选择器。

7. 【单选题】选择器 `ul > li` 与 `ul li` 的区别是？

- A. 没有区别，效果完全相同
- B. `>` 只选中直接子元素，空格选中所有后代
- C. `>` 选中所有后代，空格只选中直接子元素
- D. `>` 的优先级更高

答案：B

解析：`>`

是子代组合器，只匹配直接子元素；空格是后代组合器，匹配所有嵌套层次的后代。两者的优先级相同。

8. 【单选题】在 `box-sizing: border-box` 模式下，设置 `width: 200px; padding: 20px; border: 5px solid black`，元素实际占用的宽度是？

- A. 200px
- B. 225px
- C. 250px
- D. 245px

答案：A

解析：border-box 下 width 已包含 padding 和 border，实际占用就是 200px。content 区域被压缩为 $200 - 20 * 2 - 5 * 2 = 150\text{px}$ 。

9. 【多选题】以下哪些情况不会发生外边距折叠（Margin Collapse）？

- A. 两个相邻的块级兄弟元素
- B. 父元素和第一个子元素的顶部外边距
- C. 两个浮动元素之间
- D. 被 BFC 包裹的元素之间

答案：CD

解析：浮动元素、绝对定位元素、行内块元素、以及被 BFC（块级格式化上下文）包裹的元素不会发生外边距折叠。

10. 【判断题】在标准盒模型（content-box）中，元素的 width 属性值包含 padding 和 border 的宽度。

答案：错误

解析：标准盒模型中 width 只包含 content，padding 和 border 是额外增加的。border-box 模式下 width 才包含 content + padding + border。

第3章 现代 CSS 布局技术

11. 【单选题】以下哪个属性用于设置 Flex 项目在主轴上的对齐方式？

- A. align-items
- B. justify-content
- C. align-content
- D. flex-direction

答案：B

解析：justify-content 控制主轴（默认水平方向）上的对齐。align-items 控制交叉轴，align-content 控制多行时的行对齐。

12. 【单选题】设置 flex: 1 等同于以下哪组值？

- A. flex-grow: 1; flex-shrink: 0; flex-basis: auto
- B. flex-grow: 1; flex-shrink: 1; flex-basis: 0%
- C. flex-grow: 0; flex-shrink: 1; flex-basis: auto
- D. flex-grow: 1; flex-shrink: 1; flex-basis: auto

答案：B

解析：flex: 1 是 flex: 1 1 0% 的简写。flex-basis: 0% 意味着项目的基础大小为 0，所有空间都按 flex-grow 比例分配。

13. 【单选题】grid-template-columns: repeat(3, 1fr) 表示？

- A. 创建 3 列，每列宽度为 1fr
- B. 创建 1 列，宽度重复 3 次
- C. 创建 3 列，每列宽度为容器宽度的 1/3
- D. 创建 3 行，每行高度为 1fr

答案：A

解析：repeat(3, 1fr) 创建 3 列，每列占据 1 份剩余空间。注意 1fr 是分配剩余空间的比例单位，不是固定百分比。

14. 【多选题】以下关于 CSS Grid 的说法正确的是？

- A. Grid 是二维布局系统，可以同时控制行和列

- B. auto-fit 会拉伸项目填满容器
- C. auto-fill 会保留空轨道
- D. gap 属性定义的是网格轨道之间的间距

答案：ABCD

解析：Grid 是二维布局系统；auto-fit 拉伸现有项目填满容器；auto-fill 保留空轨道；gap 定义的是网格线（轨道）之间的间距，与项目的 margin 无关。

15. 【判断题】CSS Grid 是一维布局系统，只能处理行或列中的一个方向。

答案：错误

解析：Grid 是二维布局系统，可以同时精确控制行和列。Flexbox 才是一维的，只能控制一个方向。

第4章 响应式设计 with 媒体查询

16. 【单选题】移动优先（Mobile First）的响应式设计中，媒体查询通常使用哪个条件？

- A. max-width
- B. min-width
- C. max-height
- D. device-width

答案：B

解析：移动优先先写移动端基础样式，然后用 `min-width` 逐步增强大屏幕体验。桌面优先才使用 `max-width`。

17. 【单选题】`clamp(16px, 2vw, 24px)` 表示字体大小会？

- A. 固定为 16px
- B. 固定为 2vw
- C. 在 16px 到 24px 之间，首选值为 2vw
- D. 在 2vw 到 24px 之间，最小值为 16px

答案：C

解析：`clamp(min, preferred, max)` 返回三个值中的中间值，但限制在 `[min, max]` 范围内。即最小 16px，最大 24px，中间值按 2vw 计算。

18. 【单选题】`rem` 单位是相对于哪个元素的字体大小计算的？

- A. 父元素
- B. 当前元素
- C. `<html>`（根元素）
- D. `<body>`

答案：C

解析：`rem`（root em）始终相对于根元素 `<html>` 的字体大小。`em` 才相对于父元素。

19. 【多选题】以下哪些属于高级媒体特性？

- A. `prefers-color-scheme`
- B. `prefers-reduced-motion`

C. orientation

D. hover

答案：ABCD

解析：prefers-color-scheme 检测系统主题（light/dark）；prefers-reduced-motion 检测用户是否偏好减少动画；orientation 检测横竖屏；hover 检测设备是否支持悬停。

20. 【判断题】在移动优先策略中，基础样式应该针对大屏幕设备编写，然后通过 max-width 媒体查询适配小屏幕。

答案：错误

解析：移动优先是先写移动端样式，再通过 min-width 适配大屏幕。桌面优先才先写桌面样式，用 max-width 适配小屏幕。

第5章 JavaScript 核心基础

21. 【单选题】以下哪个变量声明具有块级作用域且不存在变量提升？

- A. var
- B. let
- C. function
- D. 以上都不对

答案：B

解析：let 和 const 具有块级作用域且不存在变量提升。var 是函数作用域且存在变量提升。function 声明存在函数提升。

22. 【单选题】 $0.1 + 0.2 === 0.3$ 的结果是？

- A. true
- B. false
- C. 取决于浏览器
- D. 会抛出错误

答案：B

解析：由于浮点数精度问题， $0.1 + 0.2$ 实际结果为 0.30000000000000004 ，不等于 0.3 。涉及金额计算应使用整数分单位。

23. 【单选题】以下哪个方法可以最准确地判断一个变量是否为数组？

- A. `typeof arr === 'object'`
- B. `arr instanceof Array`
- C. `Array.isArray(arr)`
- D. `arr.constructor === Array`

答案：C

解析：Array.isArray() 是最可靠的方法，跨 frame 也能正常工作。typeof 对数组返回 'object'；instanceof 在跨 frame 时可能失效。

24. 【多选题】以下关于 JavaScript 函数的说法正确的是？

- A. 箭头函数没有自己的 this，继承自外层作用域

- B. 函数声明存在函数提升
- C. 闭包会持有外部变量的引用
- D. 函数参数按值传递，但对象参数传递的是引用地址

答案：ABCD

解析：箭头函数没有自己的 this；函数声明会提升到作用域顶部；闭包持有外部变量引用；函数参数按值传递，对象参数传递的是引用地址的副本。

25. 【判断题】== 和 === 的区别在于 == 会进行类型转换后比较，=== 要求类型 and 值都相同。推荐始终使用 ===。

答案：正确

解析：== 进行松散相等比较，会进行类型转换；=== 进行严格相等比较，要求类型 and 值都相同。为避免类型转换带来的意外结果，推荐始终使用 ===。

第6章 ES6+ 新特性与异步编程

26. 【单选题】以下哪个运算符仅在左侧为 null 或 undefined 时返回右侧值？

- A. ||
- B. &&
- C. ??
- D. ?.

答案：C

解析：?? 是空值合并运算符，仅在左侧为 null 或 undefined 时返回右侧值。|| 会在左侧为 falsy 值（包括 0、false、''）时返回右侧值。

27. 【单选题】Promise.all([p1, p2, p3]) 的行为是？

- A. 返回最快完成的 Promise
- B. 等待所有 Promise 完成，任一失败则整体失败
- C. 等待所有 Promise 完成，无论成功失败
- D. 返回第一个成功的 Promise

答案：B

解析：Promise.all() 等待所有 Promise 完成，全部成功时返回结果数组；任一失败则立即失败，返回第一个失败的错误。

28. 【单选题】async 函数中未捕获的错误会？

- A. 静默忽略
- B. 阻塞主线程
- C. 返回一个 rejected 的 Promise
- D. 自动调用 catch

答案：C

解析：async 函数总是返回 Promise。函数内部未捕获的错误会使返回的 Promise 变为 rejected 状态，需要在调用处用 try/catch 或 .catch() 处理。

29. 【多选题】以下哪些属于 ES6+ 新增的数组方法？

- A. map

- B. find
- C. includes
- D. flat

答案：BCD

解析：find()、includes()、flat()、flatMap() 是 ES6+ 新增方法。map() 在 ES5 中就已存在。

30. 【判断题】await 会暂停 async 函数的执行，但不会阻塞主线程。

答案：正确

解析：await 会暂停 async 函数内部的执行，等待 Promise resolve，但 JavaScript 引擎可以继续执行其他代码（如事件处理），不会阻塞主线程。

第7章 React 框架基础

31. 【单选题】React 中列表渲染时 key 属性的主要作用是？

- A. 作为列表项的 ID 传递给后端
- B. 帮助 React 识别哪些元素改变了、添加了或删除了
- C. 控制列表项的显示顺序
- D. 为列表项添加 CSS 样式

答案：B

解析：key 帮助 React 识别哪些元素改变了、添加了或删除了，从而优化 Diff 算法，提高渲染性能。key 应在兄弟元素中唯一。

32. 【单选题】以下关于 useEffect 的说法正确的是？

- A. useEffect 在每次渲染后都执行
- B. `useEffect(() => {}, [])` 只在组件挂载时执行
- C. useEffect 不能返回函数
- D. useEffect 是同步执行的

答案：B

解析：`useEffect(() => {}, [])` 空依赖数组表示只在组件挂载和卸载时执行。useEffect 默认在渲染后异步执行，可以返回清理函数。

33. 【单选题】React 的 props 具有以下哪个特性？

- A. 可读写，子组件可以修改 props
- B. 只读，组件不能修改自己的 props
- C. 只能在类组件中使用
- D. 必须是一个对象

答案：B

解析：React 的 props 是只读的（Read-only），组件不能修改自己的 props。数据修改应通过调用父组件传递的回调函数，由父组件更新 state 后重新传递 props。

34. 【多选题】以下关于 React Hooks 的说法正确的是？

- A. Hooks 只能在函数组件的最顶层调用

- B. `useState` 返回一个数组，第一个元素是状态值
- C. `useRef` 的 `.current` 修改不会触发重新渲染
- D. `useEffect` 的依赖数组应包含所有在 `effect` 中使用的响应式值

答案：ABCD

解析：Hooks 必须在最顶层调用，不可在循环或条件中调用；`useState` 返回 `[state, setState]`；`useRef` 修改 `.current` 不触发渲染；`useEffect` 依赖数组应包含所有响应式值，避免闭包陷阱。

35. 【判断题】在 React 中，`false`、`null`、`undefined` 不会渲染任何内容到 DOM 中，但 `0` 和空字符串会渲染。

答案：正确

解析：React 中 `false`、`null`、`undefined` 不会渲染任何内容。但 `0` 和空字符串 `''` 是有效的 React 子节点，会渲染到 DOM 中。条件渲染时应注意 `{count && <Component />}` 在 `count` 为 `0` 时会显示 `0` 而非组件。

第8章 前端工程化与性能优化

36. 【单选题】Core Web Vitals 中衡量加载性能的指标是？

- A. INP
- B. CLS
- C. LCP
- D. FCP

答案：C

解析：LCP (Largest Contentful Paint, 最大内容绘制) 衡量加载性能，目标 < 2.5s。INP 衡量交互响应延迟，CLS 衡量视觉稳定性。

37. 【单选题】以下哪种方式可以有效防御 XSS 攻击？

- A. 使用 innerHTML 插入用户输入
- B. 对输出内容进行 HTML 实体编码
- C. 禁用浏览器的 JavaScript
- D. 使用 HTTP 代替 HTTPS

答案：B

解析：防御 XSS 的核心是对用户输入进行过滤和对输出进行编码。使用 `textContent` 替代 `innerHTML`、设置 `Content-Security-Policy`、Cookie 设置 `HttpOnly` 都是有效防御手段。

38. 【单选题】Vite 在开发阶段为什么比 Webpack 启动更快？

- A. 使用了更高效的算法
- B. 基于原生 ES Module，不需要打包
- C. 使用了更强大的服务器
- D. 缓存了所有依赖

答案：B

解析：Vite 在开发阶段基于浏览器原生 ES Module，无需打包构建，直接按需加载模块，因此启动极快。生产阶段使用 Rollup 打包。

39. 【多选题】以下哪些属于前端性能优化策略？

- A. 代码分割 (Code Splitting)

- B. 图片懒加载 (lazy loading)
- C. 使用 transform 和 opacity 实现动画
- D. 配置 HTTP 缓存 (Cache-Control)

答案：ABCD

解析：代码分割减少初始加载；图片懒加载延迟非首屏图片加载；transform 和 opacity 可利用 GPU 加速；HTTP 缓存减少重复请求。这些都是有效的性能优化策略。

40. 【判断题】Service Worker 运行在浏览器的主线程上，会阻塞 UI 渲染。

答案：错误

解析：Service Worker 运行在独立的 Worker 线程上，不会阻塞主线程和 UI 渲染。它可以拦截网络请求、管理缓存、实现离线访问等功能。